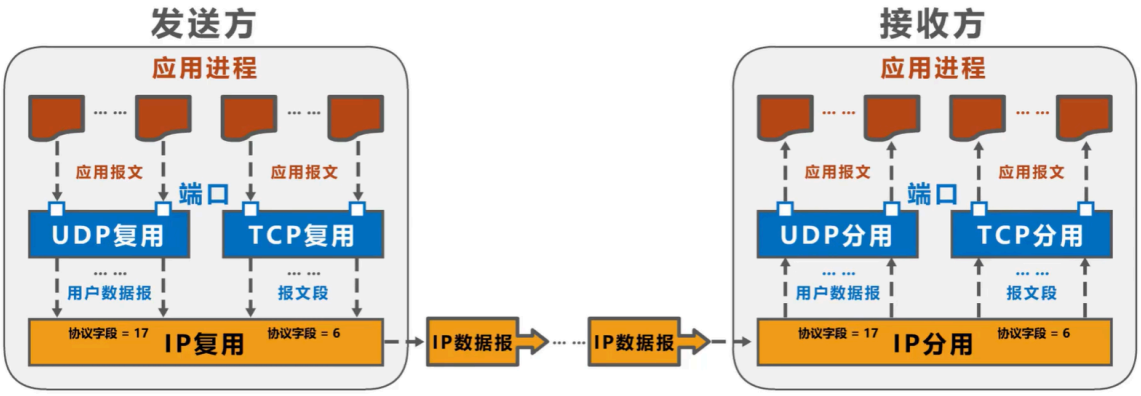
5计算机网络第五章 运输层

5.1 运输层协议概述

- **5.1.1 进程之间的通信**
  - **地位**：
    - 物理层、数据链路层、网络层：共同解决了将主机通过异构网络互连起来所面临的问题，实现了**主机到主机**的通信
    - **运输层**：用户功能中的最低层，面向通信部分的最高层
  - **作用 (考点)**：
    - **端到端通信**：运输层提供**应用进程**间的逻辑通信
    - **复用与分用**：



      - **复用 (Multiplexing)**：发送方不同的应用进程都可以使用同一个运输层协议传送数据 。
      - **分用 (Demultiplexing)**：接收方的运输层在剥去报文首部后，能够把这些数据正确交付目的应用进程 。
    - **差错检测**：运输层对收到的报文进行差错检测（IP层只检验首部，不检验数据部分） 。
    - **屏蔽作用**：向高层用户屏蔽了下面网络核心的细节（如网络拓扑、路由选择等），使应用进程看起来像是在两个运输层实体之间有一条端到端的逻辑通信信道 。
- **5.1.2 运输层的两个主要协议**
- 5.3 UDP和TCP的对比_哔哩哔哩_bilibili

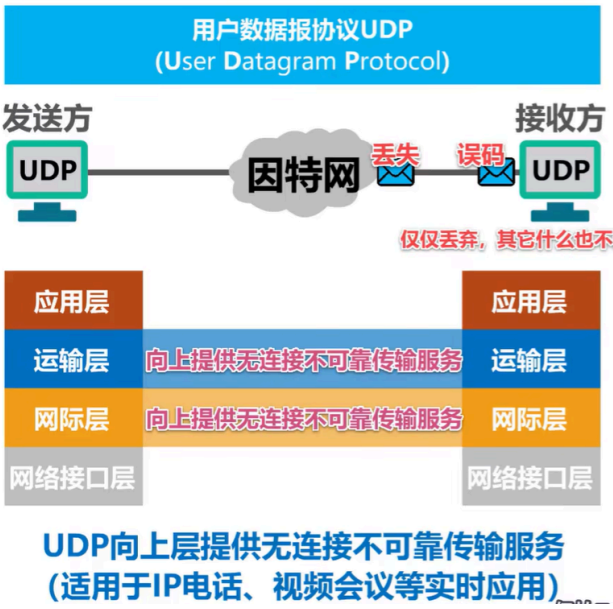
UDP

- 传送数据之前不需要先建立连接。
- 收到 UDP 报后，不需要给出任何确认。
- 不提供可靠交付，但是一种最有效的工作方式。

TCP

- 提供可靠的、面向连接的运输服务。
- 不提供广播或多播服务。
- 开销较多。

5.3 UDP和TCP的对比



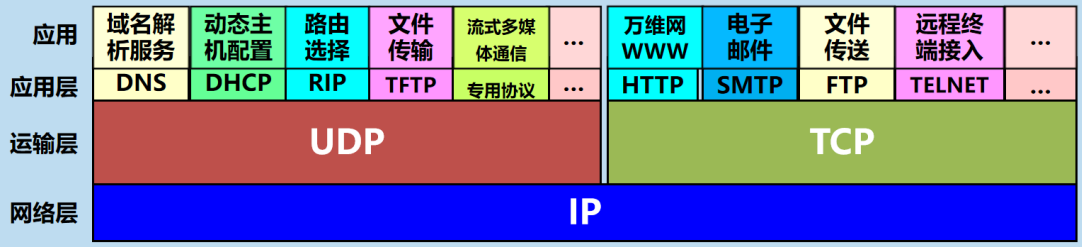
UDP向上层提供无连接不可靠传输服务
（适用于IP电话、视频会议等实时应用）



TCP向上层提供面向连接的可靠传输服务
（适用于要求可靠传输的应用，例如文件传输）

- **UDP (User Datagram Protocol)**：用户数据报协议 。
- **TCP (Transmission Control Protocol)**：传输控制协议 。

使用 UDP 和 TCP 的典型应用和应用层协议



- **两个对等运输实体传送的数据单位 (TPDU)**：
  - TCP 传送的是 **TCP 报文段 (Segment)** 。

        - UDP 传送的是 **UDP 报文** 或 **用户数据报**。
- **5.1.3 运输层的端口**
    - **端口号 (Protocol Port Number)**：
        - **作用**：为了标志本计算机应用层中的各进程。在互联网中，**不同计算机的相同端口号没有联系** 。
        - **长度**：16 位，允许有 65,535 个不同的端口号
        - **意义**：只具有==**本地意义** ==
    - ==端口的分类== (考点)**：
        1. **熟知端口 (Well-known Ports)**：
            - 范围：**0 ~ 1023**。
            - 由 IANA 分配，全球通用
        2. **登记端口 (Registered Ports)**：
            - 范围：**1024 ~ 49151**。
            - 在 IANA 登记，防止重复
        3. **短暂端口 (Ephemeral Ports)**：
            - 范围：**49152 ~ 65535**。
            - **客户进程运行时**动态选择，通信结束后收回
    - **常用的熟知端口 (重点记忆)**：
        - **FTP**: 21 (控制), 20 (数据)
        - **TELNET**: 23
        - **SMTP**: 25
        - **DNS**: 53
        - **TFTP**: 69
        - **HTTP**: 80
        - **SNMP**: 161
        - **HTTPS**: 443

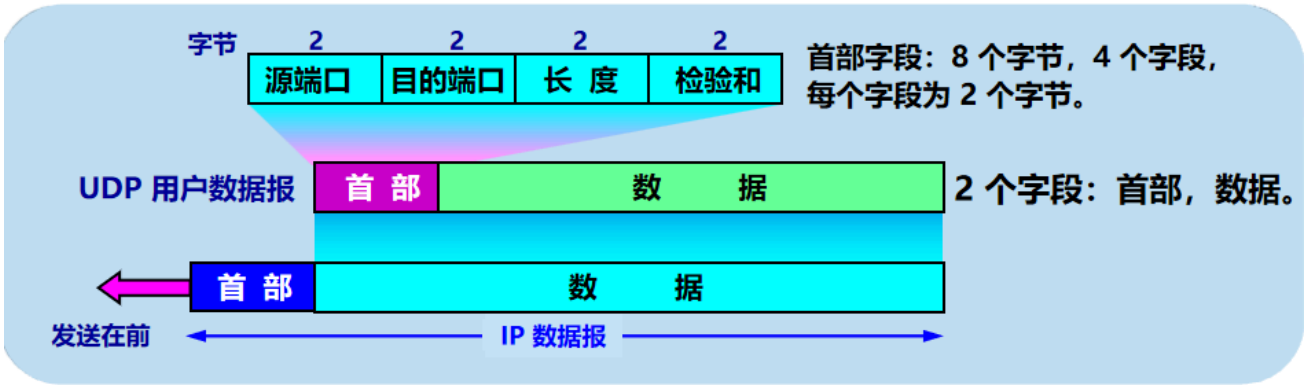
5.2 用户数据报协议 UDP

(考点：UDP特点、首部格式)

- **5.2.1 UDP 概述**
    - **功能**：在 IP 的数据报服务之上增加了 **复用、分用** 和 **差错检测** 。
    - **主要特点 (考点)**：

1. **无连接**：发送数据之前不需要建立连接。
2. **尽最大努力交付**：即不保证可靠交付。
3. **面向报文**：
    - 发送方 UDP 对应用层交下来的报文，**既不合并，也不拆分**，保留这些报文的边界 。
    - 应用层必须选择合适大小的报文（太长会导致 IP 分片，太短首部开销大） 。
4. **无拥塞控制**：网络出现的拥塞不会使源主机的发送速率降低（适合实时应用，如 IP 电话、视频会议） 。
5. **支持多种交互**：一对一、一对多、多对一、多对多 。
6. **首部开销小**：只有 **8 个字节** (TCP 是 20 字节) 。

- **5.2.2 UDP 的首部格式**



    - **结构**：数据字段 + 首部字段。
    - **首部字段 (8 字节，4 个字段，各占 2 字节) (考点)**：
        1. **源端口**：需要对方回信时选用，不需要时全 0。
        2. **目的端口**：终点交付报文时必须使用。
        3. **长度**：UDP 用户数据报的长度，最小值是 8（仅有首部）。
        4. **检验和**：检测传输中是否有错，有错就丢弃。
    - **伪首部 (Pseudo Header)**：
        - **作用**：仅仅为了计算检验和。不向下传送，也不向上递交。
        - **内容**：源 IP 地址 + 目的 IP 地址 + 0 + 17 (UDP协议号) + UDP长度。
        - **计算方法**：把伪首部、UDP首部、数据部分一起，按 **二进制反码运算求和** 28。

5.3 传输控制协议 TCP 概述

(考点：TCP 特点、套接字、连接概念)

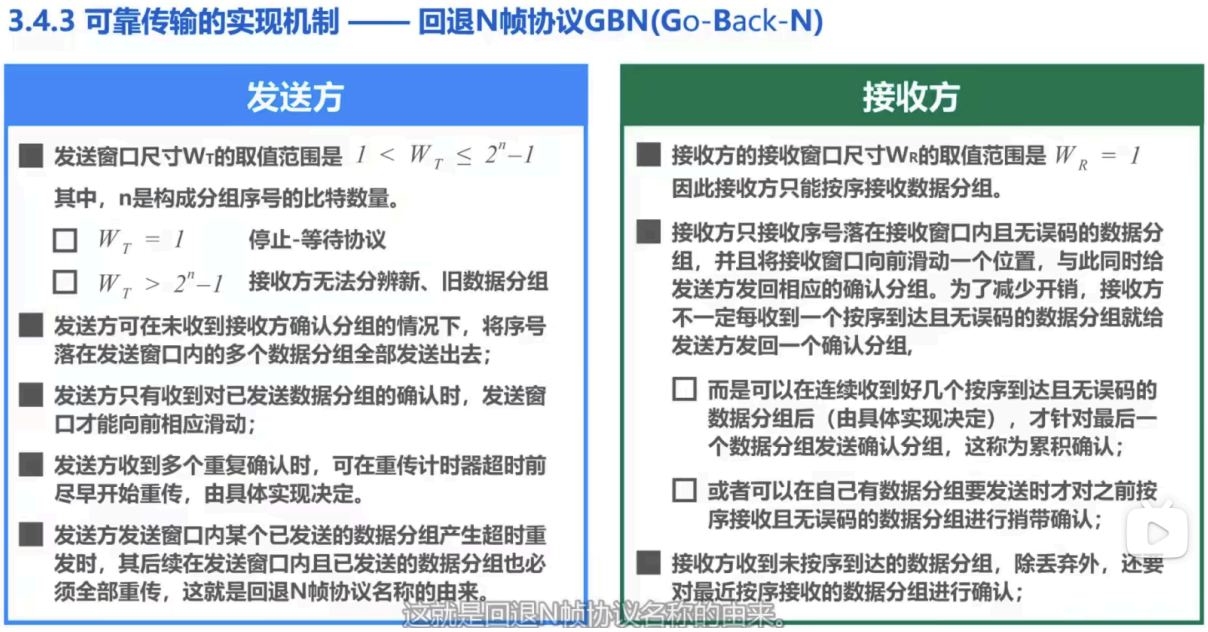
- **5.3.1 TCP 最主要的特点**
  1. **面向连接**的运输层协议 。
  2. **点对点**：每一条 TCP 连接只能有两个端点 (endpoint) 。
  3. **可靠交付**：无差错、不丢失、不重复、按序到达 。
  4. **全双工通信**：允许通信双方的应用进程在任何时候发送数据 。
  5. **面向字节流 (Byte Stream) (重点)**：
     - TCP 把应用程序交下来的数据看成仅仅是一连串**无结构的字节流**。
     - TCP 不保证接收方收到的数据块大小与发送方一致，但保证字节流完全一样。
- **5.3.2 TCP 的连接**
  - **端点**：TCP 连接的端点叫 **套接字 (socket)** 或插口 。
  - **公式**：`套接字 socket = (IP地址 : 端口号)` 。
  - **TCP 连接的定义**：`TCP 连接 ::= {socket1, socket2} = {(IP1: port1), (IP2: port2)}`
  - **注意**：同一个 IP 地址可以有多个不同 TCP 连接；同一个端口号也可以出现在多个不同TCP 连接中

5.4 ==可靠传输的工作原理

==
(考点：停止等待、ARQ、滑动窗口)

- **5.4.1 停止等待协议 (Stop-and-Wait)**
  - **原理**：每发送完一个分组就停止发送，等待对方的确认。收到确认后再发送下一个。
  - **异常处理**：
    1. **出现差错**：接收方丢弃分组，不发送任何信息。发送方通过 **超时计时器 (Timeout Timer)** 进行 **超时重传**
    2. **确认丢失**：发送方超时重传，接收方丢弃重复分组，重传确认
    3. **确认迟到**：发送方收到重复确认（丢弃），接收方收到重复分组（丢弃并重传确认）
  - **信道利用率**：
    - 公式：$U = T_D/(T_D + RTT + T)$
    - **缺点**：当往返时间 RTT 远大于分组发送时间 TD 时，信道的利用率会非常低
- **5.4.2 连续 ARQ 协议 (Automatic Repeat reQuest)**

- [计算机网络微课堂第026讲 可靠传输的实现机制 — 回退N帧协议（有字幕有背景音乐版）_哔哩哔哩_bilibili]
  - **流水线传输**：发送方连续发出多个分组，不必每发一个就停下来等待
  - **工作原理 (重点)**：
    - **发送窗口**：位于窗口内的分组可连续发送 。发送窗口大小为1~(2^n - 1)，否则接收方无法分辨新旧数据
    - **累积确认 (Cumulative ACK)**：接收方不必对每个分组发送确认，只需对 **按序到达的最后一个分组** 发送确认
      - *含义*：表示到这个分组为止的所有分组都已正确收到了。
  - **Go-back-N (回退 N)**：



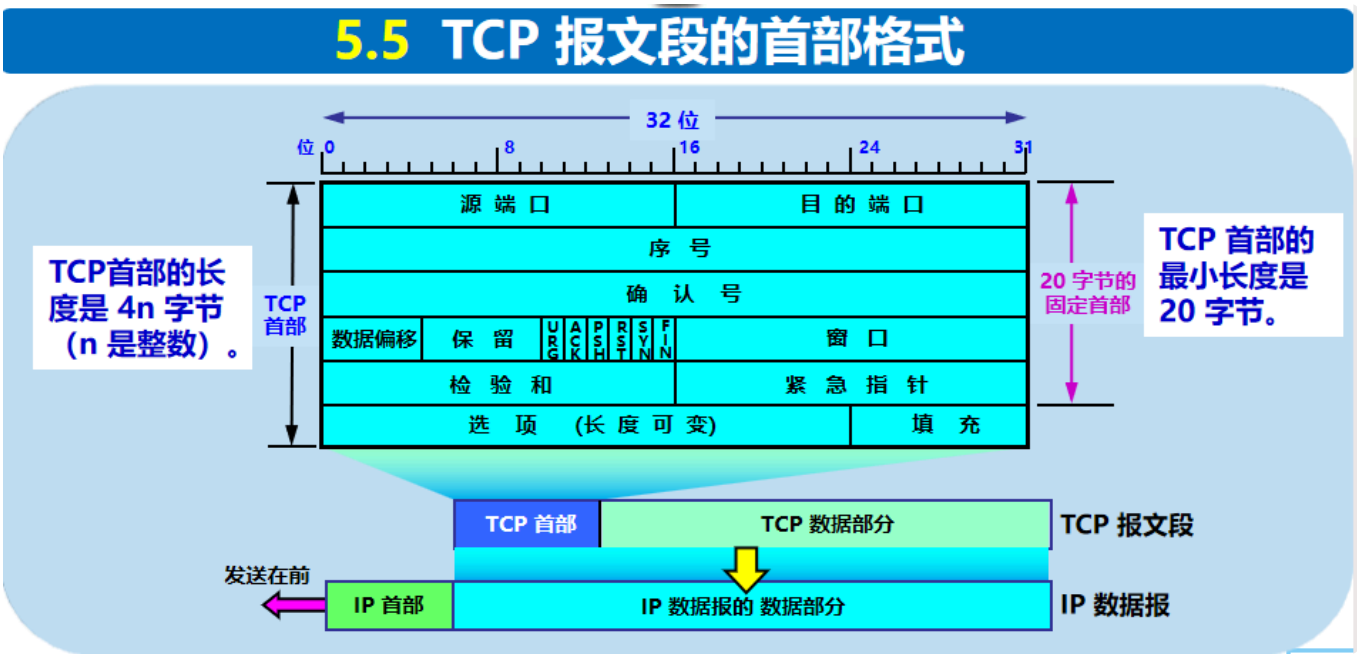
    - 如果中间某个分组丢失（如 M3 丢失，M4/M5 到达），接收方只能确认 M2。发送方超时后，必须**重传 M3 及其后面所有已发送的分组** (M4, M5)



5.5 TCP 报文段的首部格式

[5.9 TCP报文段的首部格式_哔哩哔哩_bilibili]

(考点：固定首部20字节、标志位含义、序号与确认号)



- **首部长度**：前 **20 字节** 固定，后面有 4n 字节的选项。最小 20 字节 。扩展首部最大40 字节
- **主要字段 (重点图示解析)**：
  1. **源端口、目的端口**：各 2 字节。
  2. **序号 (Sequence Number)**：4 字节。指出本报文段数据 **第一个字节** 的序号。
  3. **确认号 (Acknowledgment Number)**：4 字节。**期望** 收到对方 **下一个报文段** 的第一个字节的序号。
     - *口诀*：若确认号 = N，则表明到 N-1 为止的所有数据都已正确收到。
  4. **数据偏移 (首部长度)**：4 位。以 **4 字节** 为单位。20-60
  5. **保留**：6 bit（位），置 0。
  6. ==**控制位 (Flags) (6位) (必考)** ==：
     - **URG (Urgent)**：紧急指针有效，高优先级数据。
     - **ACK (Acknowledgment)**：连接建立后所有报文段都必须把 ACK 置 1**。
     - **PSH (Push)**：接收方尽快交付应用进程，不等待缓存填满。
     - **RST (Reset)**：连接严重差错，必须释放连接并重新建立。
     - **SYN (Synchronization)**：**同步位**。用于**建立连接**时的同步序号。
       - SYN=1, ACK=0: 连接请求。
       - SYN=1, ACK=1: 连接接受。
     - **FIN (Finish)**：**终止位**。用于**释放连接**。
  7. **窗口 (Window)**：2 字节。指明允许对方发送的数据量（基于接收方的接收能力）。
  8. **检验和**：16bit。加上 12 字节伪首部进行计算（同 UDP）。

9. **紧急指针**：2 字节。指出紧急数据的末尾位置。



紧急标志位URG：取值为1时紧急指针字段有效；取值为0时紧急指针字段无效。

紧急指针：占16比特，以字节为单位，用来指明紧急数据的长度。

当发送方有紧急数据时，可将紧急数据插队到发送缓存的最前面，并立刻封装到一个TCP报文段中进行发送。紧急指针会指出本报文段数据载荷部分包含了多长的紧急数据，紧急数据之后是普通数据。

10. **选项 (Options)**：
    - **MSS (Maximum Segment Size)**：最大报文段长度（数据字段的最大长度）。默认 536 字节 。不能太小：网络利用率降低；不能太大：开销增大；应尽可能大。
    - **窗口扩大、时间戳、SACK** 等 。

5.6 TCP 可靠传输的实现

5.7 TCP可靠传输的实现_哔哩哔哩_bilibili

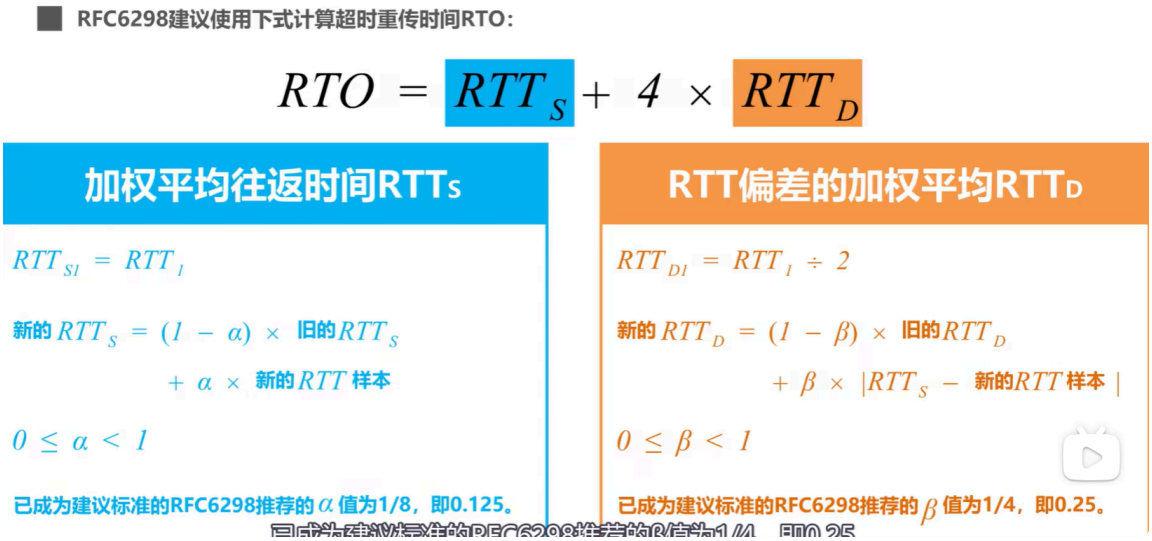
(考点：滑动窗口机制、超时时间计算)

5.7 TCP可靠传输的实现

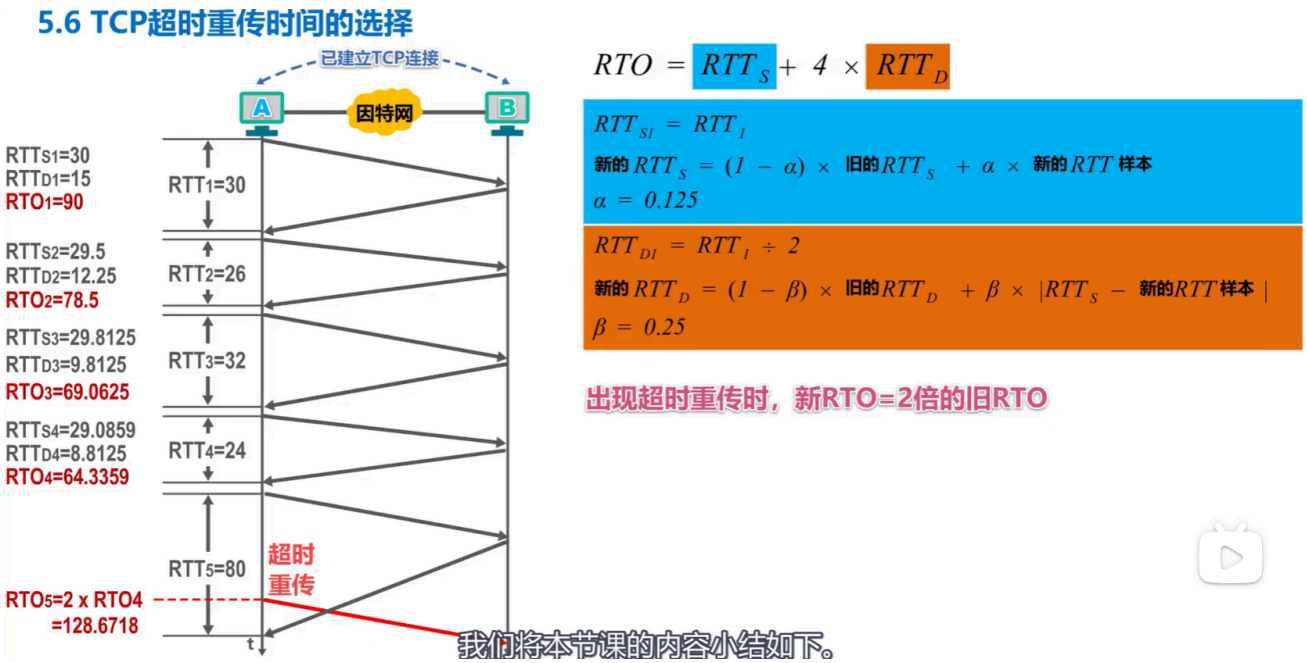
- ■ 虽然发送方的发送窗口是根据接收方的接收窗口设置的，但在同一时刻，**发送方的发送窗口并不总是和接收方的接收窗口一样大。**
  - □ 网络传送窗口值需要经历一定的时间滞后，并且这个时间还是不确定的。
  - □ 发送方还可能根据网络当时的拥塞情况适当减小自己的发送窗口尺寸。
- ■ 对于**不按序到达的数据应如何处理**，TCP并无明确规定。
  - □ 如果接收方把不按序到达的数据一律丢弃，那么接收窗口的管理将会比较简单，但这样做对网络资源的利用不利，因为发送方会重复传送较多的数据。
  - □ TCP通常对不按序到达的数据是先临时存放在接收窗口中，等到字节流中所缺少的字节收到后，再**按序交付上层的应用进程。**
- ■ TCP要求接收方必须有**累积确认和捎带确认机制**，这样可以减小传输开销。接收方可以在合适的时候发送确认，也可以在自己有数据要发送时把确认信息顺便捎带上。
  - □ **接收方不应过分推迟发送确认**，否则会导致发送方不必要的超时重传，这反而浪费了网络的资源。

    TCP标准规定，确认推迟的时间不应超过0.5秒。若收到一连串具有最大长度的报文段，则必须每隔一个报文段就发送一个确认[RFC 1122]。
  - □ 捎带确认实际上并不经常发生，因为大多数应用程序很少同时在两个方向上发送数据。
- ■ **TCP的通信是全双工通信。**通信中的每一方都在发送和接收报文段。因此，每一方都有自己的发送窗口和接收窗口。在谈到这些窗口时，一定要弄清楚是哪一方的窗口。

- **5.6.1 ==以字节为单位的滑动窗口==**
  - **发送窗口**：
    - $P3 - P1$ = 发送窗口 (通知窗口)。
      112
    - $P2 - P1$ = 已发送但未收到确认。
    - $P3 - P2$ = 允许发送但尚未发送 (可用窗口) 53。
  - **机制**：
    - 发送窗口由 **rwnd (接收窗口)** 和 **cwnd (拥塞窗口)** 决定。
    - 发送方收到确认后，窗口**前移**
    - **缓存**：发送缓存存放准备发送和已发送未确认的数据；接收缓存存放按序到达未读取和未按序到达的数据
- **5.6.2 ==超时重传时间的选择==**（复杂）
- 5.6 TCP超时重传时间的选择_哔哩哔哩_bilibili
  - **RTT (Round-Trip Time)**：往返时间。
  - **加权平均往返时间** RTT_S (Smoothed RTT) 56：
    - $RTT_S = (1 -) \times (\text{旧 } RTT_S) + \ \times (\text{新 } RTT \text{ 样本})$。
    - 推荐 $= 012$。
  - **超时重传时间 RTO (Retransmission Time-Out)** 57：
    - $RT = RTT_S + 4 \times RTT_D$ (RTT_D 是 RTT 偏差的加权平均)。



  - **Karn 算法**：
    - 若报文段重传，**不采用**其 RTT 样本（因为无法区分确认是针对原报文还是重传报文）。
    - **修正**：报文每重传一次，RTO 增大一倍



- **5.6.3 选择确认 SACK**
  - **作用**：接收方收到不连续的字节块时，通过 SACK 选项告诉发送方“哪些块收到了”，让发送方只重传丢失的数据 59。
  - **实现**：TCP 首部选项中报告字节块的左边界和右边界 60。

5.7 TCP 的流量控制

5.7、5.8、5.9 https://chat.deepseek.com/share/fz4a8jkfbyenuk2xdm

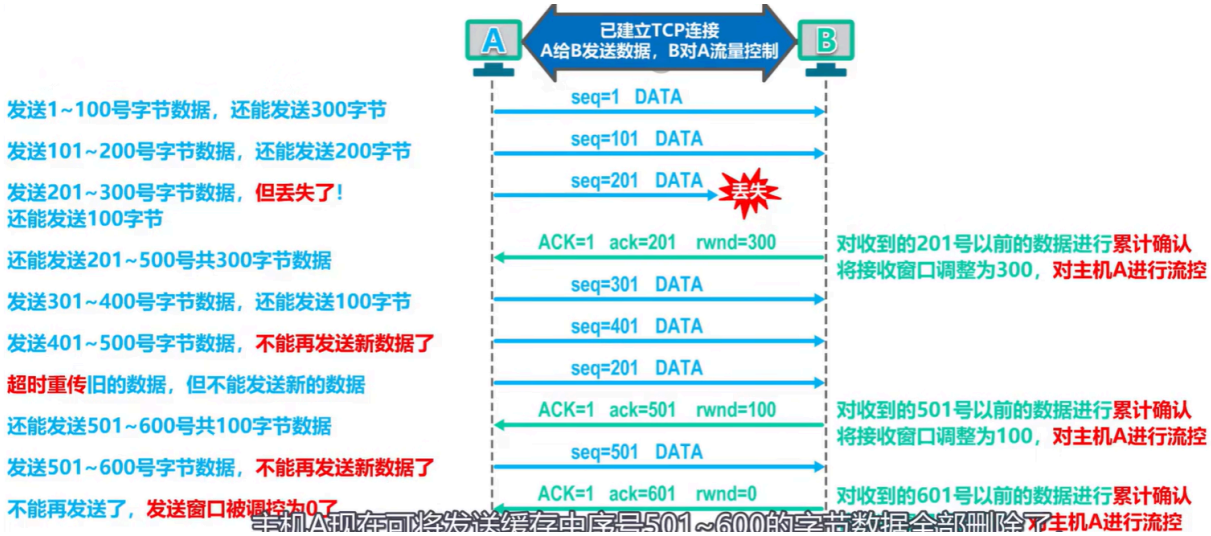
5.4 TCP的流量控制_哔哩哔哩_bilibili

流量控制解决的是“**接收方**跟不跟得上”的问题

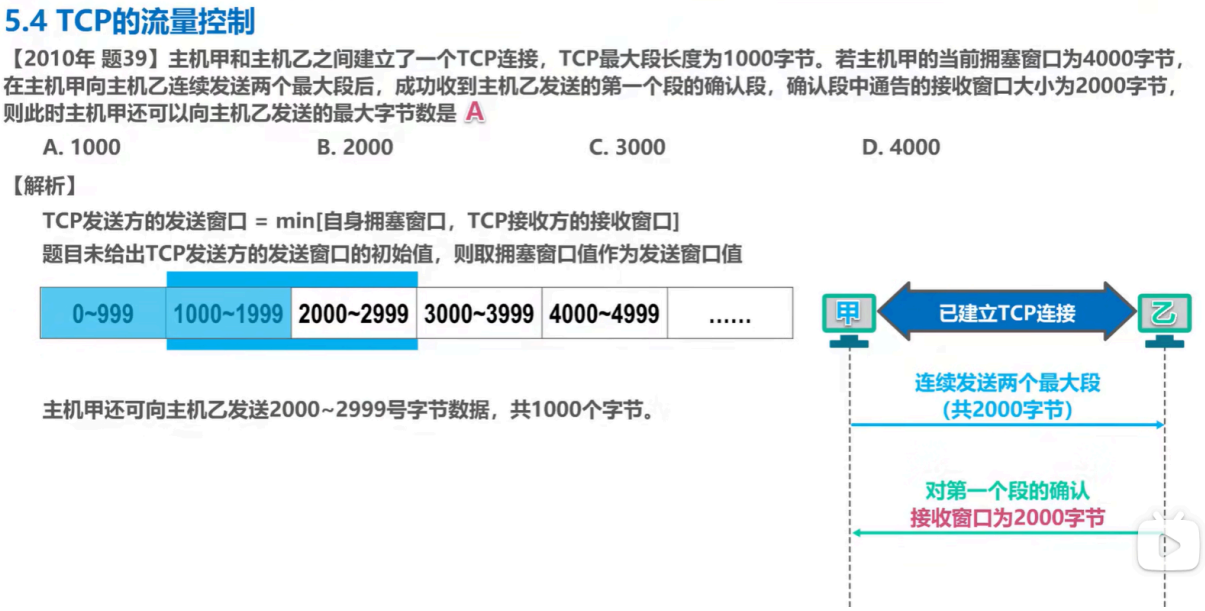
(考点：rwnd、零窗口死锁)

- **5.7.1 利用滑动窗口实现流量控制**
  - **定义**：让发送方的发送速率不要太快，使接收方来得及接收
  - **机制**：接收方在 ACK 中通知 **rwnd (接收窗口)** 大小。发送方的发送窗口不能超过 rwnd

- ACK是返回消息，ack是希望收到下一个序号为，rwnd是接收窗口大小



- **死锁问题 (Deadlock)**：
  - 场景：B 向 A 发送 `rwnd = 0`，A 停止发送。后来 B 有了缓存，发送 `rwnd = 400`，但该报文**丢失**。A 等待 B 的通知，B 等待 A 的数据，形成死锁
  - **解决方法：持续计时器 (Persistence Timer)**。
    - 一方收到零窗口通知就启动计时器。到期后发送 **零窗口探测报文段** (1 字节)，对方需确认并给出当前窗口值



- **5.7.2 TCP 的传输效率**
  - **Nagle 算法**：解决 **发送方糊涂窗口综合症** (每次发 1 字节，开销巨大)。
    - 策略：先发第一个字节，收到确认后再把缓存中剩余数据组装成一个报文段发送
  - **Clark 算法**：解决 **接收方糊涂窗口综合症** (接收方每次只腾出 1 字节缓存)。
    - 策略：接收方在缓存有一半空闲或足够容纳一个 MSS 之前，通告窗口为 0 。

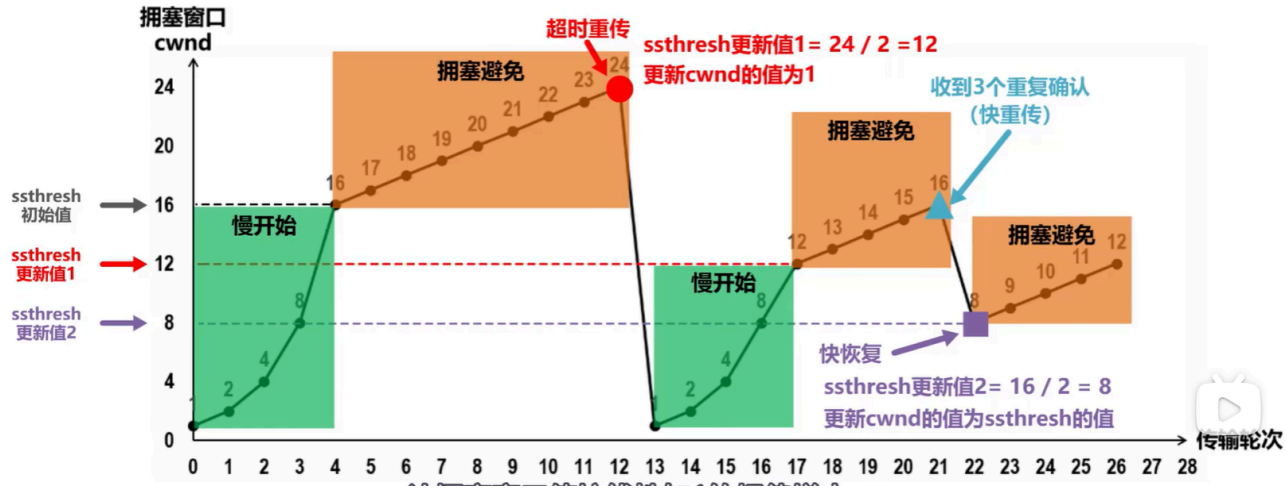
5.8 TCP 的拥塞控制

5.5 TCP的拥塞控制_哔哩哔哩_bilibili

拥塞控制解决的是“**整条通信线路（网络）** 堵不堵车”的问题，是整个网络交通的事。

(考点：四大算法、拥塞窗口变化图)

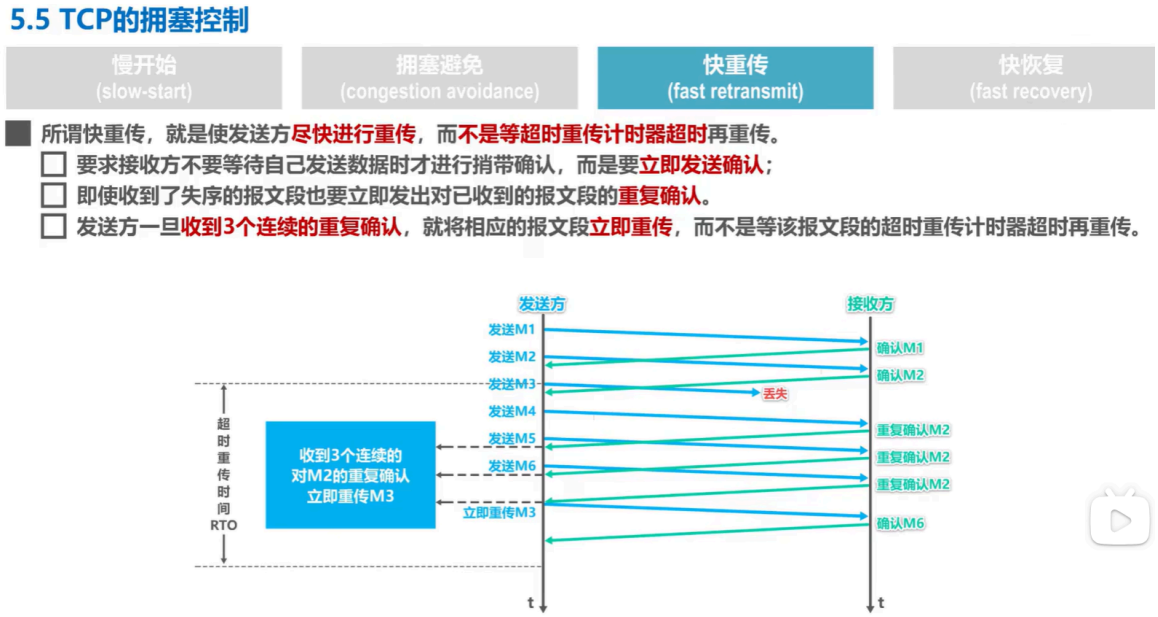
- **5.8.1 拥塞控制的一般原理**
  - **定义**：防止过多的数据注入到网络中，使网络中的路由器或链路不致过载 。
  - **区别**：
    - **流量控制**：点对点（端到端），解决接收端接收能力不足。
    - **拥塞控制**：全局性，涉及网络中所有的主机和路由器 。
  - **拥塞判断**：重传定时器超时（丢包）。
- **5.8.2 TCP 的拥塞控制方法 (重点必考)**



  - **发送窗口上限**：$Mn[rnd, nd]$。
  - **四种算法**：
    1. **慢开始 (Slow Start)**：
       - `cwnd` 初始为 1 (MSS)。
       - 每收到一个 ACK，`cwnd` 加倍 (指数增长：1, 2, 4, 8...)。
       - 直到达到 **慢开始门限 ssthresh**。
       - 发送窗口将拥塞窗口作为发送窗口，即rwnd=cwnd
    2. **拥塞避免 (Congestion Avoidance)**：
       - `cwnd >= ssthresh` 时执行。
       - 每经过一个 RTT，`cwnd` 增加 1 (加法增大，线性增长)。
       - **网络拥塞时 (超时)**：
         - `ssthresh` 设为当前 `cwnd` 的一半。即12

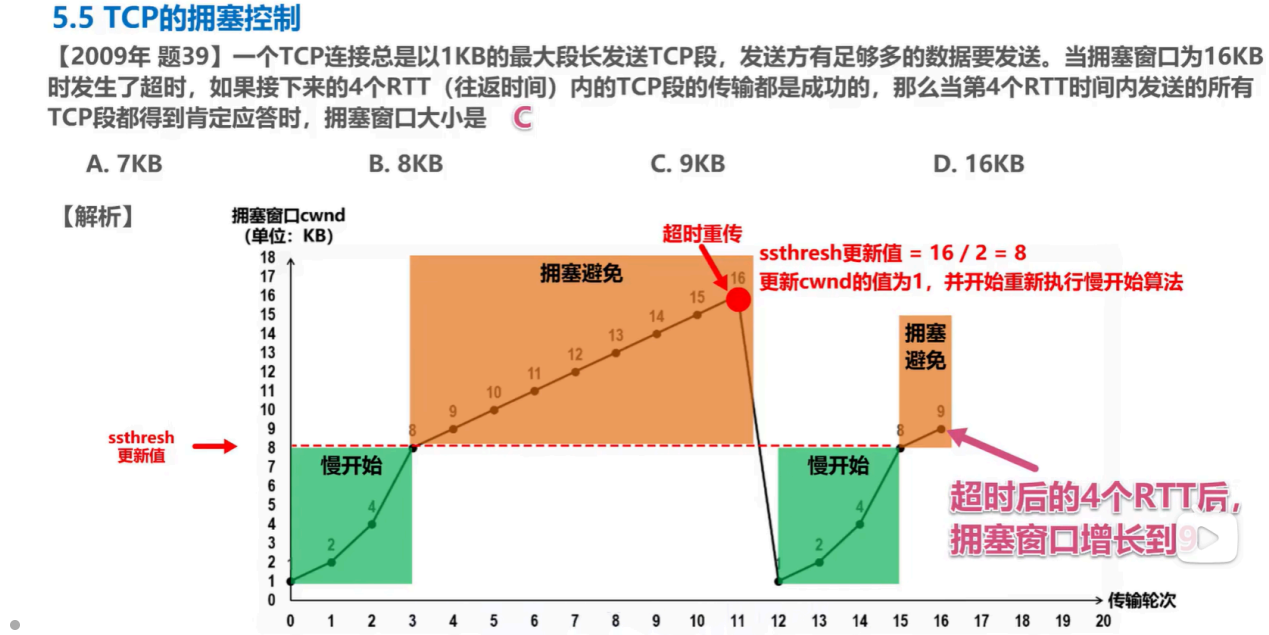
- `cwnd` 重置为 1。
- 重新执行慢开始。

3. **快重传 (Fast Retransmit)**：



- 接收方收到失序报文段，立即发送**重复确认**。
- 发送方一连收到 **3 个重复确认**，就立即重传丢失的报文段，不必等待超时 。

4. **快恢复 (Fast Recovery)**：
   - 当收到 3 个重复确认时（并未发生严重拥塞）：
   - `ssthresh` 减半。
   - `cwnd` 设置为新的 `ssthresh` (而不是 1)。
   - 执行**拥塞避免** (线性增长)。



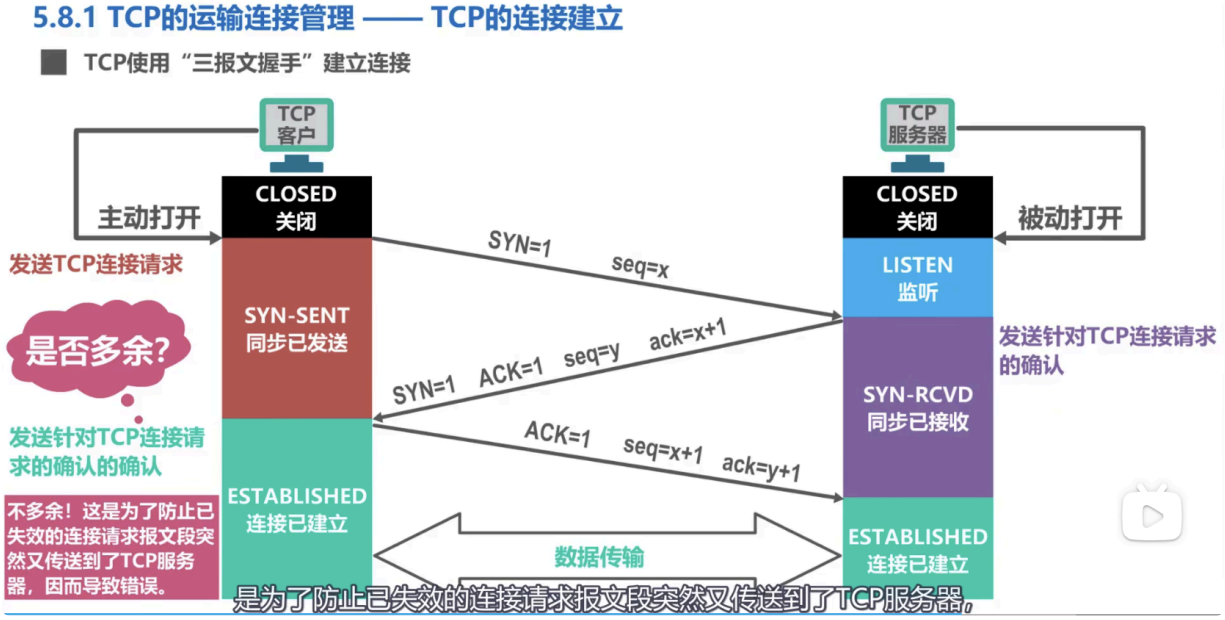
- **5.8.3 主动队列管理 AQM**

- **RED (Random Early Detection)**：随机早期检测。在队列未满时就随机丢弃分组，避免全局同步 。

5.9 ==TCP 的运输连接管理==

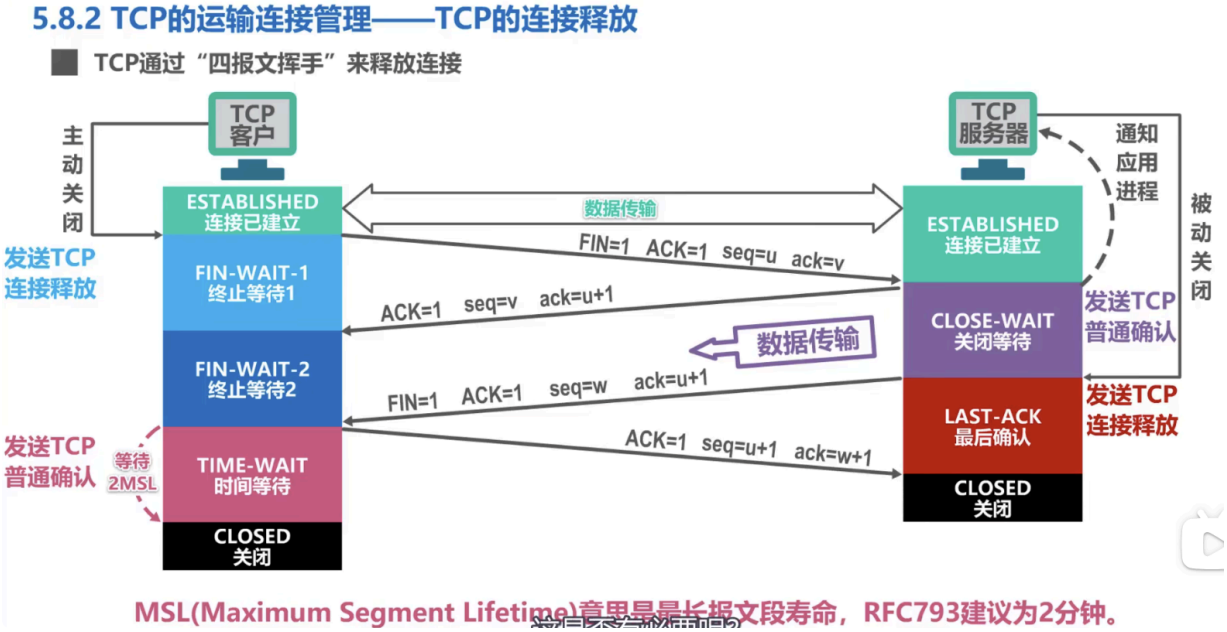
(考点：三次握手、四次挥手、状态机)

- **5.9.1 TCP 的连接建立 (三次握手) (重点图示)**
- 5.8.1 TCP的运输连接管理—TCP的连接建立_哔哩哔哩_bilibili
  - **目的**：同步双方的序号 (ISN)，协商参数 (如 MSS、窗口扩大)。
  - **过程**：



    1. **Client -> Server (SYN)**：
       - `SYN = 1`, `seq = x`。
       - 客户机进入 **SYN-SENT** 状态。
    2. **Server -> Client (SYN + ACK)**：
       - `SYN = 1`, `ACK = 1`, `seq = y`, `ack = x + 1`。
       - 服务器进入 **SYN-RCVD** 状态。
    3. **Client -> Server (ACK)**：
       - `ACK = 1`, `seq = x + 1`, `ack = y + 1`。
       - 客户机进入 **ESTABLISHED** 状态。服务器收到后也进入 **ESTABLISHED**。
  - **为什么是三次？**：防止已失效的连接请求报文段突然又传送到了服务端，产生错误连接。

- **5.9.2 TCP 的连接释放 (四次挥手) (重点图示)**
- [5.8.2 TCP的运输连接管理—TCP的连接释放_哔哩哔哩_bilibili]
  - **过程：**



    1. **A -> B (FIN)**：`FIN=1, seq=u`。A 进入 **FIN-WAIT-1**。
    2. **B -> A (ACK)**：`ACK=1, seq=v, ack=u+1`。B 进入 **CLOSE-WAIT**。A 收到后进入 **FIN-WAIT-2**。
       - *此时连接处于半关闭状态，A 不能发数据，但能收数据。*
    3. **B -> A (FIN + ACK)**：`FIN=1, ACK=1, seq=w, ack=u+1`。B 进入 **LAST-ACK**。
    4. **A -> B (ACK)**：`ACK=1, seq=u+1, ack=w+1`。A 进入 **TIME-WAIT**。B 收到后 **CLOSED**。
  - **TIME-WAIT 状态：**
    - A 必须等待 **2MSL** (最长报文段寿命) 后才关闭。
    - **原因：**
      1. 保证 A 发送的最后一个 ACK 报文段能够到达 B (如果丢失，B 会超时重传 FIN)。
      2. 防止“已失效的连接请求报文段”出现在本连接中。



- TCP服务器进程每收到一次TCP客户进程的数据，就重新设置并启动**保活计时器**（2小时定时）。

- 若保活计时器定时周期内未收到TCP客户进程发来的数据，则**当保活计时器到时后，TCP服务器进程就向TCP客户进程发送一个探测报文段**，以后则每隔75秒钟发送一次。若一连发送10个探测报文段后仍无TCP客户进程的响应，TCP服务器进程就认为TCP客户进程所在主机出了故障，接着就关闭这个连接。

- **5.9.3 TCP 的有限状态机**

